

Problem Set 2 - Interpolation

Fred Milhome

2026-03-01

To reproduce the results in this report, get the replication files in [my GitHub repository pset2 folder](#) and follow the README.md.

Question 1: Life Expectancy Interpolation

Below is a plot of the life expectancy data points and the linear interpolation function constructed from those points. The code prints the interpolated value for 1996 as 69,23 years.

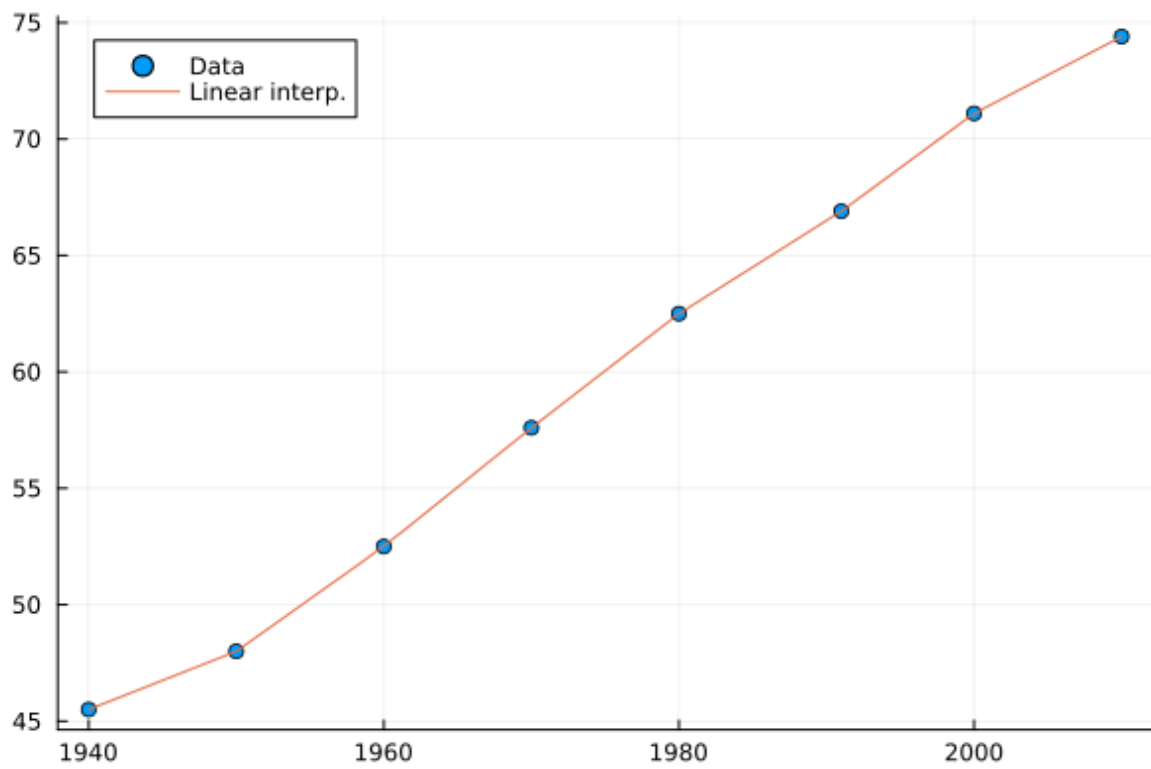


Figure 1: Life expectancy linear interpolation

Questions 2 and 3: Interpolation Comparison

To leverage on the DRY principle, I chose to answer both questions simultaneously, which gives us a comparison of the combinations of interpolation method and grid spacing, and allows us to see how the fit for each of those combinations improve for each grid granularity choice.

Items a-b, e

In the `compare_interp` script, these steps were executed in a way that the sample is drawn only once for each distribution, and a helper function `generate_grid` was created to generate the grid for each of the two spacing choices. The main helper function is `compare_interp`, which evaluates the three interpolation methods for each combination of distribution, spacing, and grid granularity choice, and returns the MSE and timing results for each of those combinations. For better efficiency, in the benchmarking step, the three interpolation methods are parallelized, and 10.000 samples are drawn for each of those methods to get a more stable estimate of the timing results. The script also runs the main loop to solve the exercises.

Items c and d

Panel Plots by Grid Size

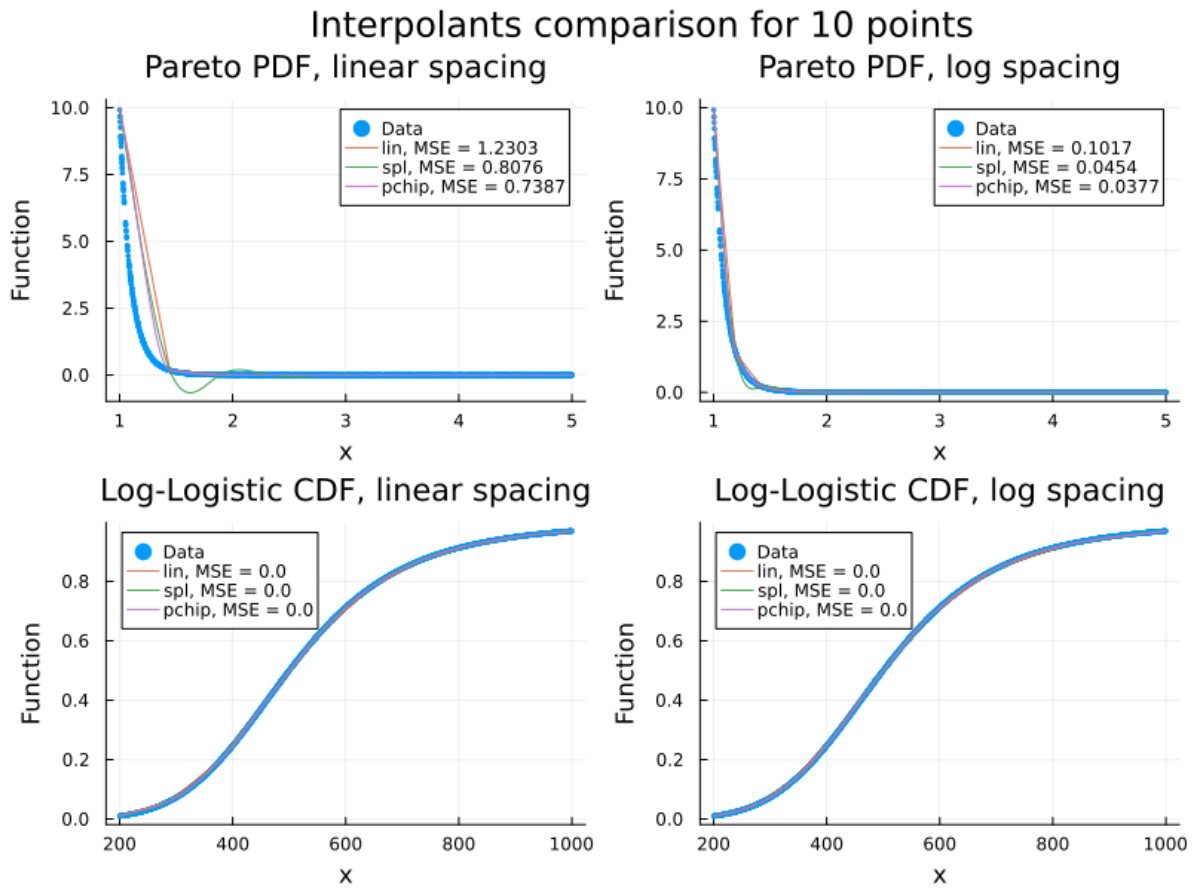


Figure 2: Fit comparison for 10 points in the grid

Note that the fit for the Log Logistic CDF is very good even for the coarsest grid, independently of spacing choice or interpolation method, since the function doesn't have much curvature.

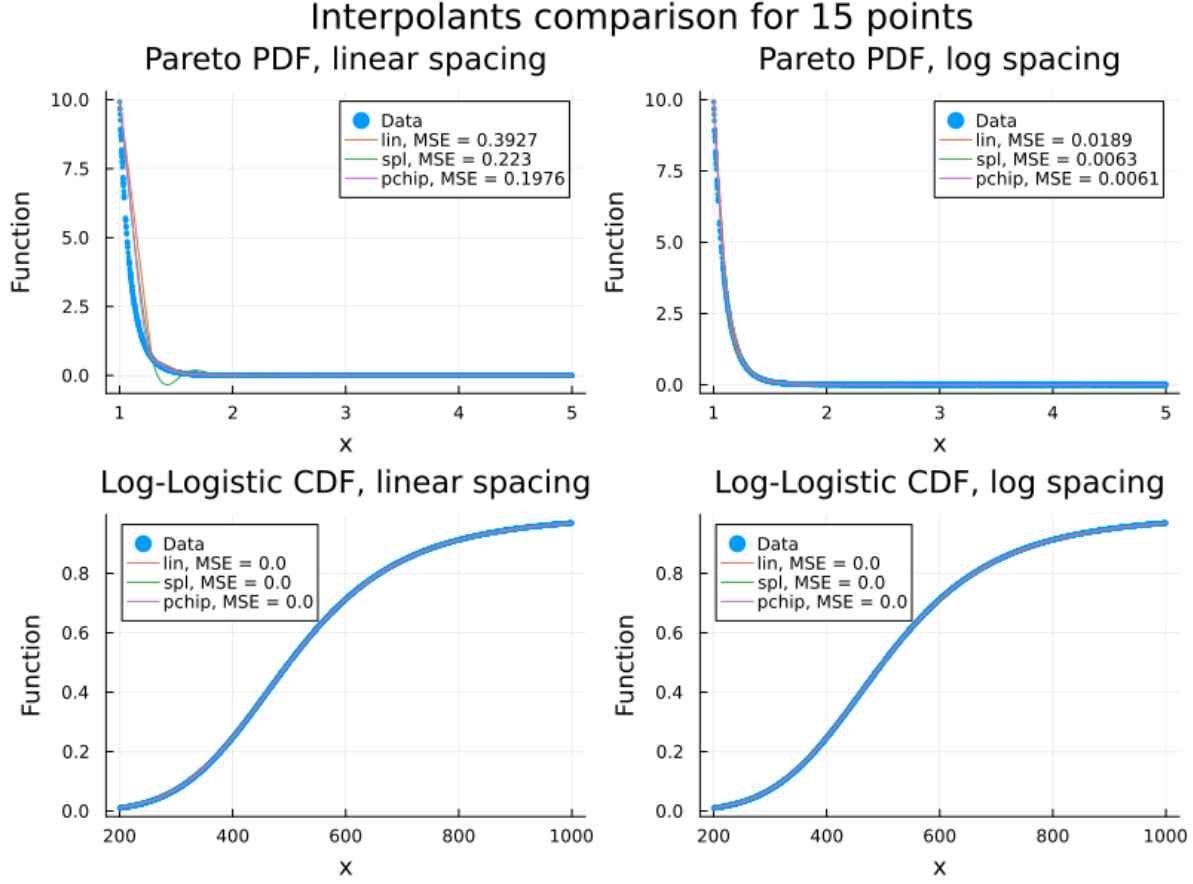


Figure 3: Fit comparison for 15 points in the grid

For the Pareto PDF, the fit is very poor for the first two coarsest grids, and for different reasons depending on the interpolation method.

With linear interpolation, a straight line hardly represents well the curvature in the left region of the function. This is greatly improved with log spacing, which places more points in that region. With the cubic spline, we face the problem of overshooting, as this method places constraints on the continuity of the first and second derivatives, which leads to having distant points (the ones from the flatter region of the function) influence the fit in the left region, which has a lot of curvature. We see that PCHIP is able to preserve monotonicity, while marginally improving the MSE.

In general, the fit improves the most when we use log spacing, which is good if one has to stick to a coarse grid in an application with a very curved function. Then, the choice of interpolation method can be important for desired properties of the fit, such as monotonicity or desired continuity of derivatives.

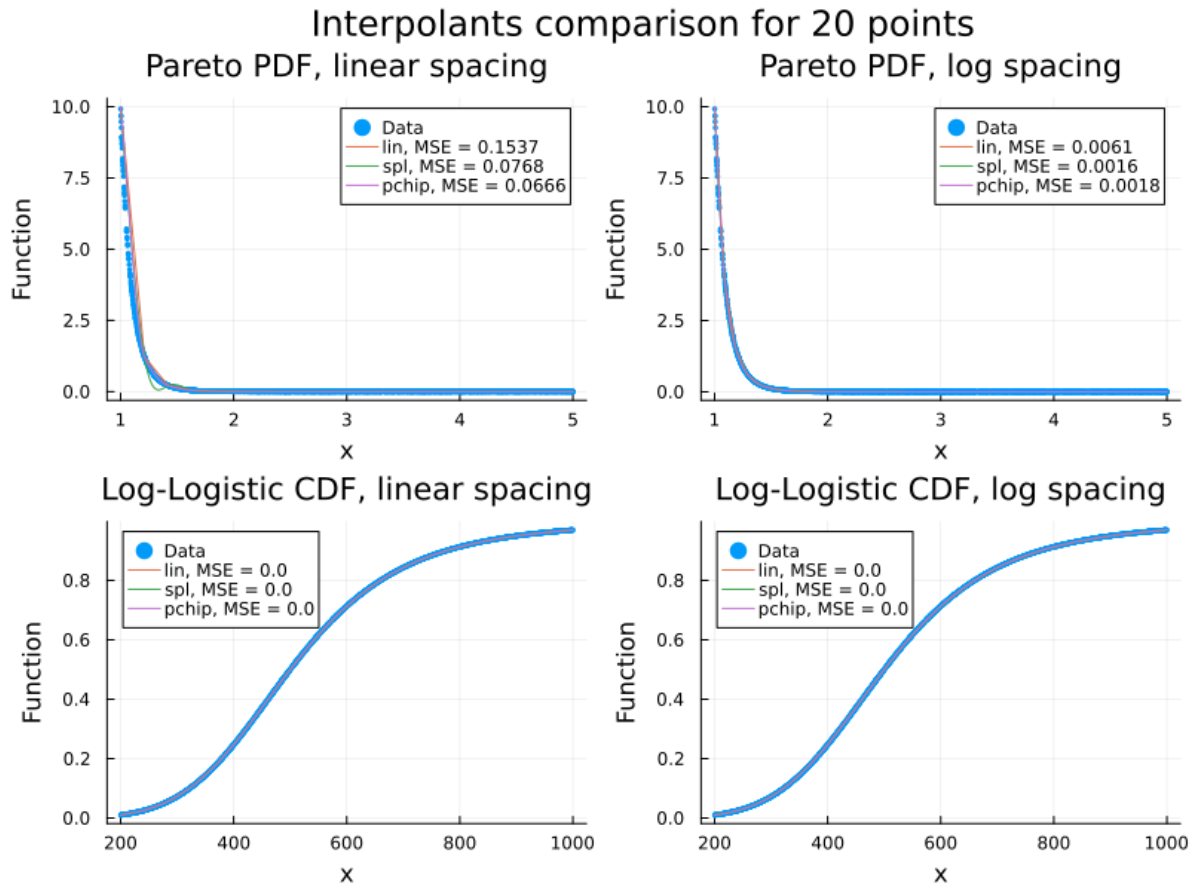


Figure 4: Fit comparison for 20 points in the grid

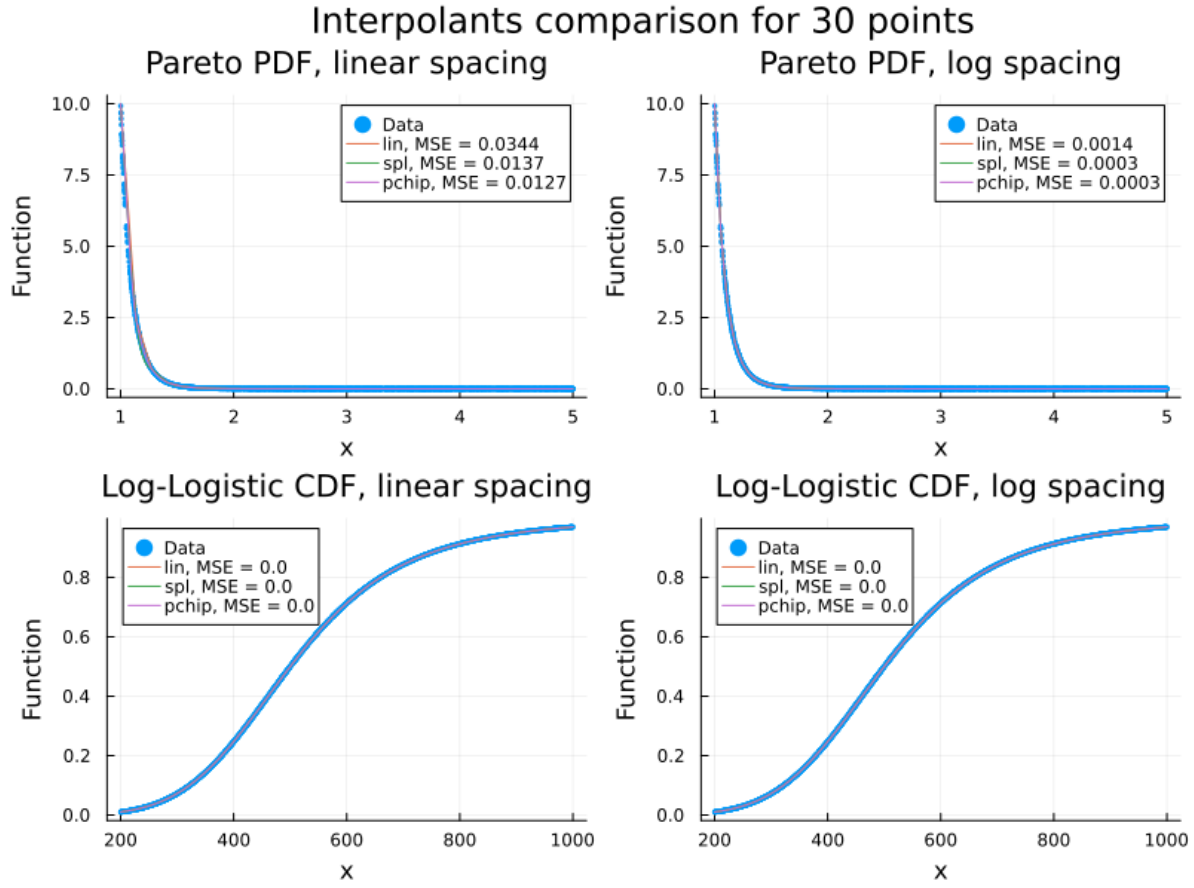


Figure 5: Fit comparison for 30 points in the grid

As we increase the number of points in the grid, the fit improves for all combinations of interpolation method and spacing choice, but the differences between them become less pronounced.

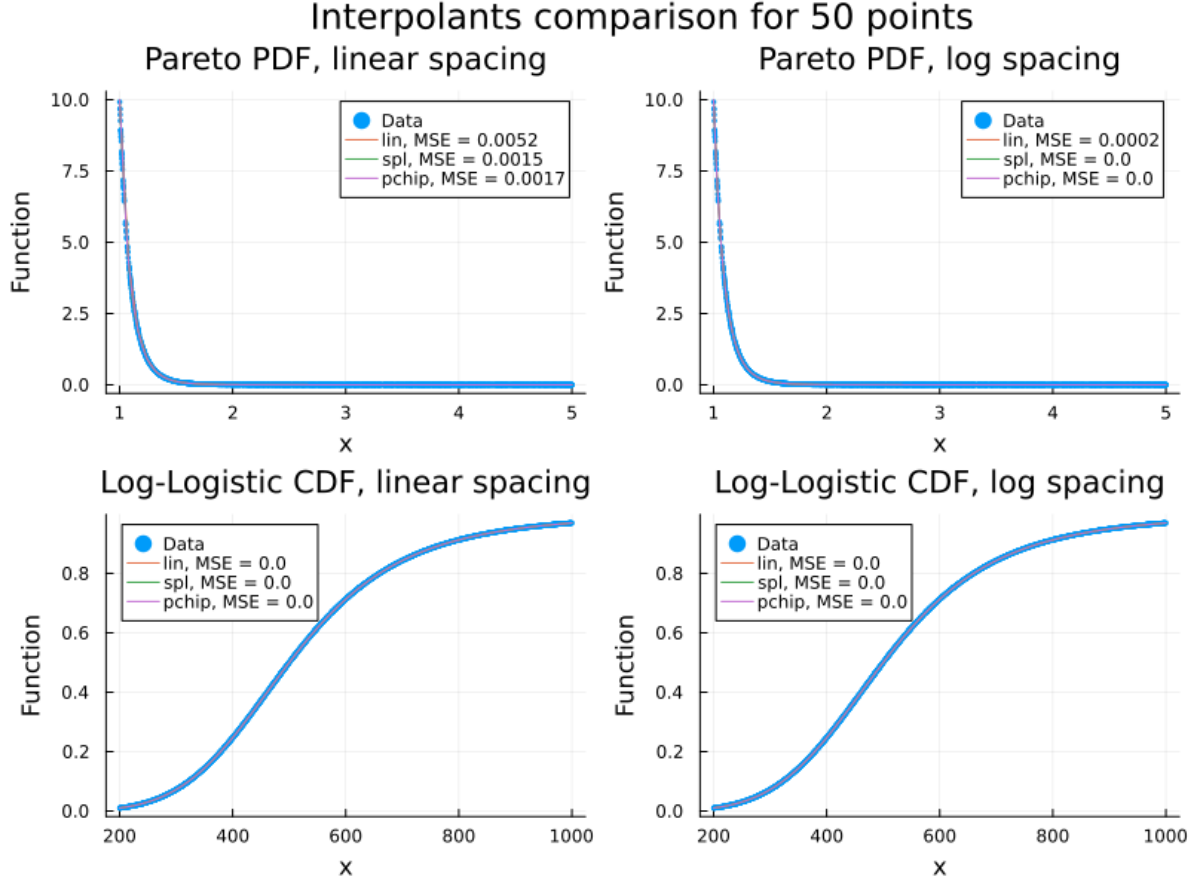


Figure 6: Fit comparison for 50 points in the grid

MSE and Timing Tables

Now I stick to analyzing the MSE and timing results for each of the combinations of interpolation method and spacing choice, for each of the two distributions. The tables below show that, not surprisingly, the MSE decreases as we increase the number of points in the grid, and that log spacing generally leads to a lower MSE than linear spacing, especially for the Pareto PDF. The timing results show that linear interpolation is generally faster than cubic spline and PCHIP as expected. In this implementation, cubic spline is approximately 10x slower than PCHIP, which is a bit surprising, but it can be due to package implementation differences.

Another interesting observation is that the timing results seem to increase with log spacing for the two more complex interpolation methods.

Pareto PDF — Linear Spacing

Grid Size (t)	Linear		Spline		PCHIP	
	MSE	Time (ns)	MSE	Time (ns)	MSE	Time (ns)
10	1.23	169	0.8076	5502	0.7387	681
15	0.3927	154	0.223	6074	0.1976	534
20	0.1537	179	0.07684	6827	0.06659	732
30	0.0344	256	0.0137	7893	0.01268	1307
50	0.005169	254	0.00149	9052	0.001719	1741

Pareto PDF — Log Spacing

Grid Size (t)	Linear		Spline		PCHIP	
	MSE	Time (ns)	MSE	Time (ns)	MSE	Time (ns)
10	0.1017	157	0.0454	5622	0.03774	417
15	0.01885	169	0.006251	5876	0.006069	590
20	0.006126	203	0.001622	6393	0.001839	777
30	0.001372	216	0.0002545	7011	0.0003378	1011
50	0.0001774	220	2.16e-05	9375	3.182e-05	1296

For the Log-Logistic CDF, the MSE is very low for all combinations of interpolation method and spacing choice, as we saw earlier. Timing results are also very similar for this function to the ones for the Pareto PDF.

Log-Logistic CDF — Linear Spacing

Grid Size (t)	Linear		Spline		PCHIP	
	MSE	Time (ns)	MSE	Time (ns)	MSE	Time (ns)
10	2.584e-05	152	1.625e-07	5705	2.216e-06	435
15	4.393e-06	166	2.058e-08	5726	1.475e-07	602
20	1.292e-06	192	4.451e-09	6788	2.419e-08	728
30	2.426e-07	211	5.725e-10	7953	2.285e-09	894
50	2.857e-08	184	4.285e-11	10419	1.291e-10	1462

Log-Logistic CDF — Log Spacing

Grid Size (t)	Linear		Spline		PCHIP	
	MSE	Time (ns)	MSE	Time (ns)	MSE	Time (ns)
10	2.244e-05	231	1.33e-07	9240	1.025e-06	810
15	3.807e-06	191	1.433e-08	7851	7.517e-08	601
20	1.158e-06	219	3.137e-09	6770	1.28e-08	857
30	2.158e-07	203	3.845e-10	6875	1.261e-09	989
50	2.665e-08	248	2.629e-11	9337	7.082e-11	1481

Question 4: Root Finding

For this exercise, I implemented the three root finding methods in the `RootFinding` submodule building upon the functions in the lecture notes, with small defensive programming improvements.

```
include(joinpath(@__DIR__, "code", "SubModules", "RootFinding.jl"))
# Function to interpolate
f(x) = x3 + 2x + 5

# Derivative of the function for Newton-Raphson
df(x) = 3x2 + 2
```

I naively search an interval which brackets the root looking outward. For this function with a single root, this works fine, but we have to be careful with multiple roots.

```
# Bracketing the root, starting with a small interval and outward looking
a = -1
b = 1

while f(a) * f(b) > 0
    a -= 1
    b += 1
end
println("Bracketing interval: [", a, ", ", b, ""])
```

Bracketing interval: [-2, 2]

Then I run the three methods to get the number of iterations to achieve the desired tolerance.

```
bisection_root, bisection_iters = RootFinding.bisection(f, a, b, tol=1e-8)
secant_root, secant_iters = RootFinding.secant(f, a, b, tol=1e-8)
newton_root, newton_iters = RootFinding.newton_raphson(f, df, (a+b)/2, tol=1e-8)
```

Number of iterations by method
=====

Bisection:	26
Secant:	7
Newton-Raphson:	7

Root = -1.3283